

DATA STRUCTURES

LECTURE 01 (INTRODUCTION TO CLASSES)

1

POINTERS

- A **pointer** is a variable whose value is the address of another variable.
- Like any variable or constant, you must declare a pointer before you can work with it.

```
type *var-name;
```

USING POINTERS

There are few important operations

1. Define a pointer variables
2. Assign the address of a variable to a pointer
3. Access the value at the address available in the pointer variable.

CONTD.

Defining a Pointer Variable

```
int *iptr;
```

iptr can hold the address of an int

Pointer Variables Assignment:

```
int num = 25;
```

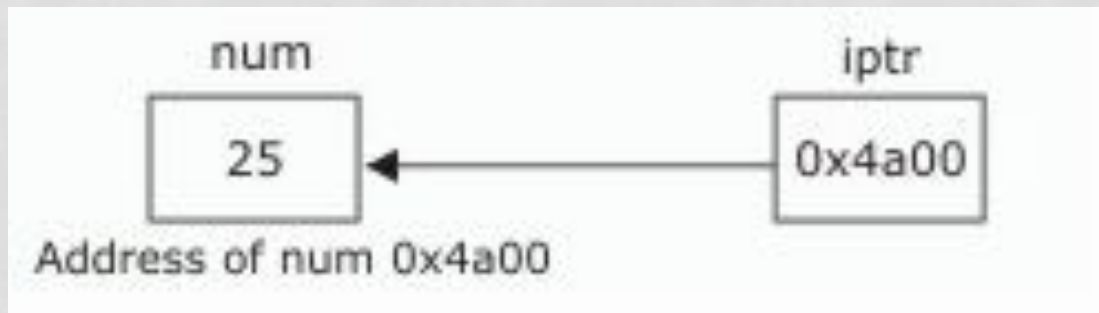
```
iptr = &num;
```

CONTD.

To access num using iptr and indirection operator *

```
cout << iptr;    // prints 0x4a00
```

```
cout << *iptr;   // prints 25
```

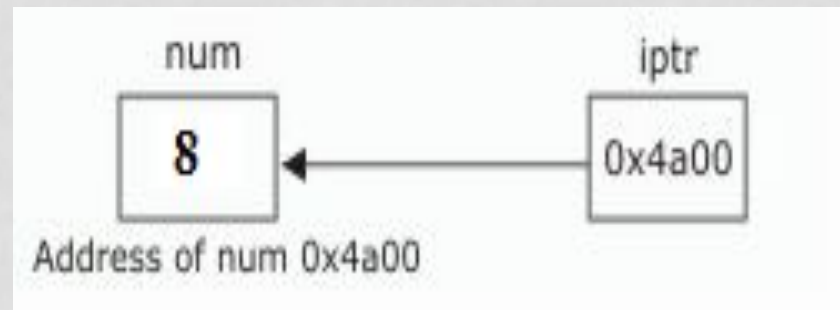


MANIPULATING DATA WITH POINTERS

Value of memory location towards which pointer is pointing can be changed using pointers

`*ptr_nam=val;`

For example `*iptr=8;`



EXAMPLE 1

```
void main
{
int * ptr, *foo, *h;
int val=9;
ptr=&val;
foo=ptr;
*foo=10;
int z=*ptr;
cout << val<<" "<<z;
}
```

?? What will be the output?

10 10

EXAMPLE 2

```
void main  
{  
  int *p;  
  int val=9;  
  p=val;  
  cout<<p;  
}
```

?? Error in the code?

EXAMPLE 3

```
void main{  
int a = 5, b = 10;  
int *p1, *p2;  
p1 = &a;  
p2 = &b;  
*p1 = 10;  
p1 = p2;  
*p1 = 20;  
cout<<a<<" "<<b;}  

```

?? What will be the output?

10 20

POINTER ARITHMETIC

- **Some mathematical operations may be performed on pointers.**
- The ++ and -- operators may be used to increment or decrement a pointer variable.
- An integer may be added to or subtracted from a pointer variable. This may be performed with the + and - operators, or the += and -= operators.
- A pointer may be subtracted from another pointer.

POINTER ARITHMETIC

- There are four arithmetic operators that can be used on pointers: ++, --, +, and –
 - Ptr++ (move one address forward)
 - Ptr - - (move one address backward)
 - Ptr+i (move i locations forward)
 - Ptr-i (move i locations backward)

CLASS EXERCISE

Assume `pint` is a pointer variable. Is each of the following statements valid or invalid? If any is invalid, why?

- A) `pint++;`
- B) `--pint;`
- C) `pint /= 2;`
- D) `pint *= 4;`
- E) `pint += x; // Assume x is an int.`

COMPARING POINTERS

- **If one address comes before another address in memory, the first address is considered “less than” the second. C++’s relational operators may be used to compare pointer values.**

Pointers may be compared by using any of C++’s relational operators:

> < == != >= <=

EXAMPLE



NOTE: Comparing two pointers is not the same as comparing the values the two pointers point to. For example, the following `if` statement compares the addresses stored in the pointer variables `ptr1` and `ptr2`:

```
if (ptr1 < ptr2)
```

The following statement, however, compares the values that `ptr1` and `ptr2` point to:

```
if (*ptr1 < *ptr2)
```

INITIALIZING POINTER TO 0

It is always a good practice to assign the pointer NULL to a pointer variable in case you do not have exact address to be assigned.

```
void main ()  
{  
    int *ptr = NULL;  
}
```

CONTD.

- If all unused pointers are given the null value you can avoid the accidental misuse of an uninitialized pointer.
- Many times, uninitialized variables hold some junk values and it becomes difficult to debug the program.

```
if(ptr){ // succeeds if p is not null }
```

```
if(!ptr) { // succeeds if p is null }
```


POINTER RULES SUMMARY

- A pointer stores a reference to its pointee. The pointee, in turn, stores something useful.
- The dereference operation on a pointer accesses its pointee. A pointer may only be dereferenced after it has been assigned to refer to a pointee. Most pointer bugs involve violating this one rule.
- Allocating a pointer does not automatically assign it to refer to a pointee. Assigning the pointer to refer to a specific pointee is a separate operation which is easy to forget.
- Assignment between two pointers makes them refer to the same pointee which introduces sharing.

STATIC ALLOCATION OF MEMORY

In the static memory allocation,

- The amount of memory to be allocated is predicted and pre known.
- Memory is allocated during the compilation itself.

All the declared variables declared normally, are allocated memory statically.

EXAMPLE

Variables deletes as the scope ends

```
void main  
{  
  int a=9;  
  cout<<sqr(a);  
}
```

```
int sqr(int val)  
{  
  return val*val;  
}
```

DYNAMIC MEMORY

Variables declared and used locally inside a function definition are destroyed, when declared variables within a function scope are needed to be used by other functions without creating an overhead by copying these variables via function's return value.

DYNAMIC ALLOCATION OF MEMORY

- Allocate memory when required
- Use it by initializing to some value
- Deallocate the memory when no more required

ALLOCATE MEMORY

```
ptrtype* ptrname;  
pvalue = new ptrtype;
```

Example:

```
void main  
{  
  int*p;  
  p=new int;  
}
```

INITIALIZE MEMORY

```
*ptrname=value;
```

Example:

```
void main  
{  
  int*p;  
  p=new int;  
  *p=9;  
}
```

DE-ALLOCATE MEMORY

```
delete ptrname;
```

Example:

```
void main  
{  
  int*p;  
  p=new int;  
  *p=9;  
  cout<<*p;  
  delete p;  
}
```


STATIC VS. DYNAMIC MEMORY

Example 1

Static	Dynamic
<pre>void main { int a,b; cin>>a>>b; int*p=&a; int*q=&b; cout<<a+b; }</pre>	<pre>void main { int *p,*q; p=new int; q=new int; cin>>*p>>*q; cout<<*p+*q; del p;del q; }</pre>

DANGLING POINTERS

- The delete operator does not delete the pointer
- it takes the memory being pointed to and returns it to the heap
- It does not even change the contents of the pointer

Since the memory being pointed to is no longer available (and may even be given to another application), such a pointer is said to be **dangling**

CONTD.

Return memory to the heap before un dangling the pointer

What's Wrong with the Following?

```
ptr = NULL;  
delete ptr;
```

MEMORY LEAKS

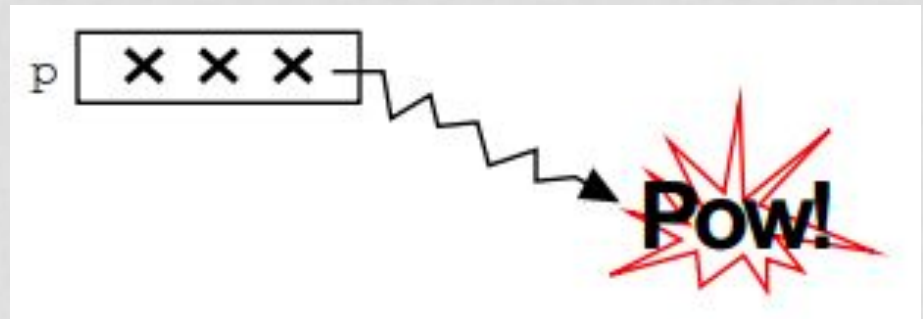
Memory *leaks* when it is allocated using the new operator but not returned using the delete operator

Example??

BAD POINTER

```
void BadPointer() {  
    int* p; // allocate the pointer, but not the pointee  
    *p = 42; //dereference is a serious runtime error  
}
```

What happens at runtime when the bad pointer is dereferenced??



DYNAMIC ARRAYS

In C++ size of array must be defined for e.g

```
int arr[9];
```

or

```
const int size=9;
```

```
int arr[size];
```

If we want to take size of array from user at runtime
dynamic array is used

CONTD.

```
void main()
{int size; int *ptr;
cin>>size;
ptr=new int[size];
for (int i=0;i<size;i++)
{
cin>>ptr[i] // consecutive locations thus access //
them
}}
```

DE-ALLOCATING ARRAY

To deallocate a dynamic array, use this form:

```
delete [] name_of_pointer;
```


DYNAMICALLY RESIZING AN ARRAY

Here is an example using an integer array. Let's say this is the original array:

```
int * list = new int[size];
```

I want to resize this so that the array called list has space for 5 more numbers (presumably because the old one is full).

FAQS

- Email datastructuresalgorithm@gmail.com
 - Queries
- LMS
 - Lab Task submission
 - Assignment submission
 - Project submission

TEXT BOOK READINGS

- Interlude 1 , “Data Abstraction and Problem solving with C++: Walls and Mirrors,” 6th Ed, Frank M. Carrano.
- Interlude 2 , “Data Abstraction and Problem solving with C++: Walls and Mirrors,” 6th Ed, Frank M. Carrano.

FAQS

- MS Teams
Queries

- LMS
Lab Task submission
Assignment submission
Project submission

REFERENCE

- Chapter 16 , C How to Program by Deitel & Deitel, 9th Ed.